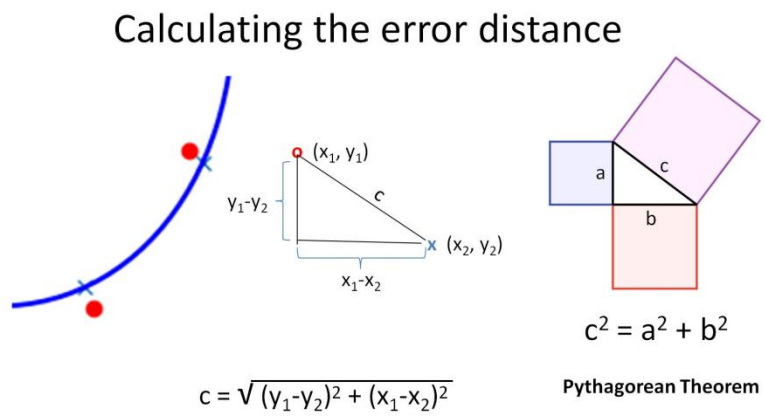
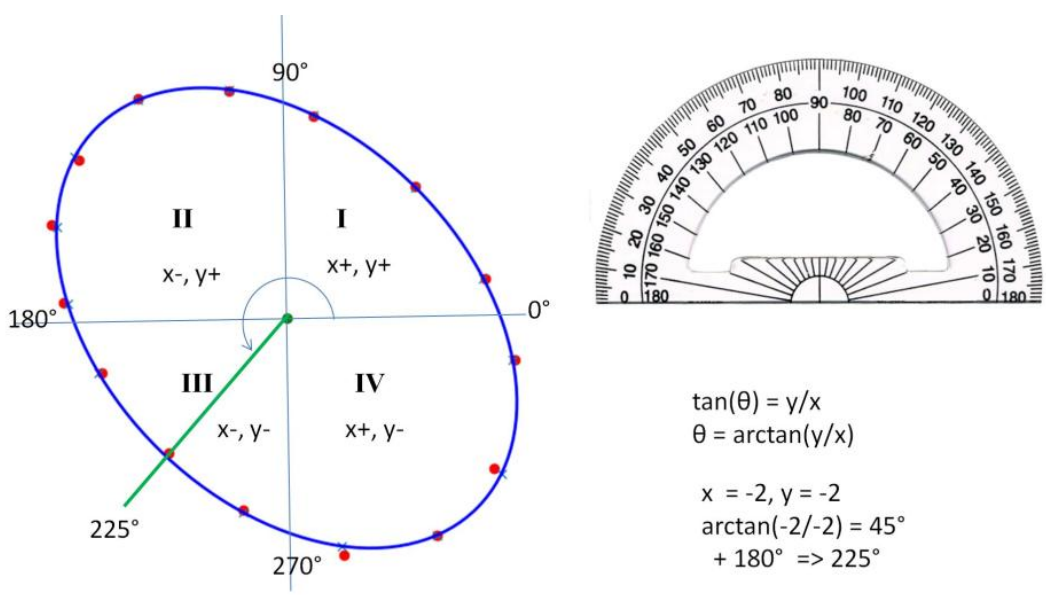
Goodness of fit for ellipses.

This Python program implements a goodness of fit measure for ellipses that is independent of the number of sample points and that is scaled to the size of the ellipse to allow comparison of the fitting errors for ellipses of different sizes. The program calculates the angle of each point relative to the center of the ellipse, and then finds the intersection point of a line at that angle on the ellipse.

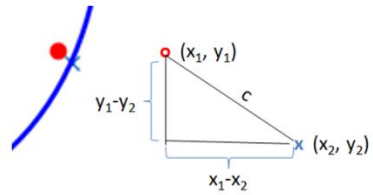


The Python program adjusts the values obtained from the trigonometric functions depending on the quadrant relative to the center of the ellipse. As you can see in the following example, the angle obtained by the **arctan** function for a point in the third quadrant is the same as the angle for the first quadrant. To get the correct value, we add 180 degrees. The **atan2** function gives negative values for the third and fourth quadrants, and this is adjusted by adding 360 degrees.



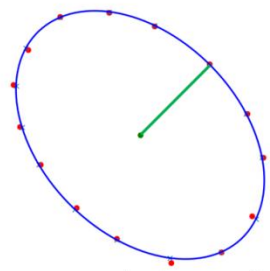
Defining the Fitting Error

$$\text{error_distance} = c = \sqrt{(y_1 - y_2)^2 + (x_1 - x_2)^2}$$



$$\text{average_error} = \frac{1}{n}\sum_{i=1}^{n} c_i$$

$$\text{fitting_error} = \frac{\text{average_error} \times 100}{\text{semiminor_axis}}$$

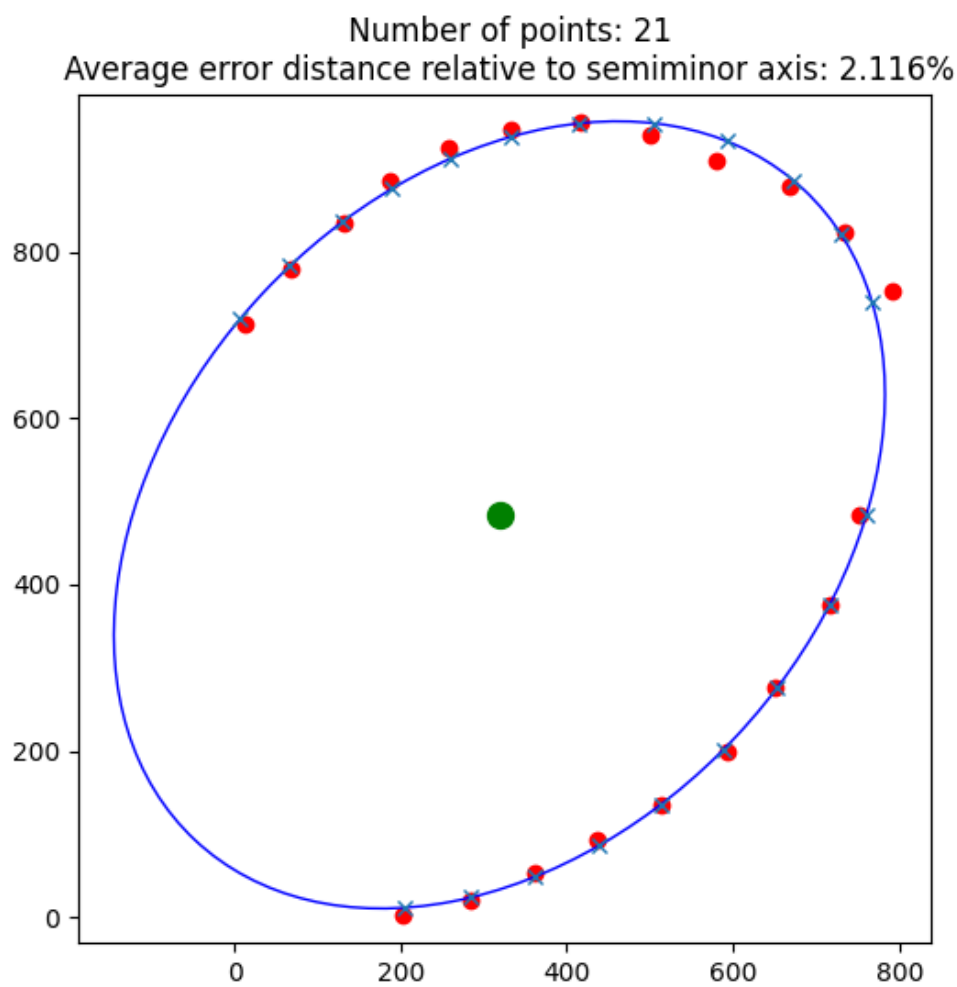


semiminor axis
of the ellipse

Standard measures of fitness, like the Mean Squared Error, do not provide meaningful comparisons for ellipses of different sizes. The error measure has to be independent of the number of sample points and the size of the ellipse. The error distance is the distance between the observed point and a point on the ellipse. The average error is the sum of all the error distances divided by the number of points. The fitting error is the percentage of the average error relative to the semiminor axis of the ellipse, which is half the width of the ellipse. Dividing by the semiminor axis scales the average error to the ellipse to allow comparisons of the fitting errors for ellipses of different sizes.

There are two examples in the code, but one has been commented out. These are the results for Example #2:

Number of points = 21
center (x,y): 318.991, 483.750
semimajor axis = 535.6987, semiminor axis = 389.9497
angle of rotation phi = 0.8176 (46.845 deg.)
Average error distance = 8.2523
Average error distance relative to semiminor axis = 2.1163%



Evolution of the Python code

The code to calculate the point where a line from the center of the ellipse intersects the curve of the ellipse is provided by the function `ellipse_line_intersection()` The code was developed with the aid of several AI models. The initial version was a numerical solution from Gemini that depended on an initial guess. The numerical solution failed in some test cases where the guess was not optimal. The

current code uses an algebraic solution from Copilot with comments added by Grok, and it works for all test cases.

The code rotates the line to align the ellipse with the coordinate axes. The parametric form of the line after rotation is:

$$(x', y') = t(x_1', y_1')$$

Substituting the line into the ellipse equation and rearranging gives:

$$\frac{(tx_1')^2}{a^2} + \frac{(ty_1')^2}{b^2} = 1$$

$$t^2 \left(\frac{x_1'^2}{a^2} + \frac{y_1'^2}{b^2} \right) = 1$$

$$\left(\frac{x_1'^2}{a^2} + \frac{y_1'^2}{b^2} \right) t^2 - 1 = 0$$

This is a quadratic equation of the form $At^2 + Bt + C = 0$, where B = 0 and C = -1.

The original code from Copilot was written in general form with the following values, but the terms in parentheses simplify to zero because the line goes through the center of the ellipse.

$$B = 2 \left(\frac{(X_c \cos\phi + Y_c \sin\phi - X_c \cos\phi - Y_c \sin\phi) x_1'}{a^2} + \frac{(X_c \sin\phi - Y_c \cos\phi - X_c \sin\phi + Y_c \cos\phi) y_1'}{b^2} \right)$$

$$C = \frac{(X_c \cos\phi + Y_c \sin\phi - X_c \cos\phi - Y_c \sin\phi)^2}{a^2} + \frac{(X_c \sin\phi - Y_c \cos\phi - X_c \sin\phi + Y_c \cos\phi)^2}{b^2} - 1$$